

ゲームプレイ向けコンポーネント

このページでは、MIDI 入力を Unity のゲームロジックに結び付けるための高レベルコンポーネントについて説明します。

名前空間: `jp.kshoji.unity.midi`

配置: `Assets/MIDI/Scripts/Gameplay/`

概要

コンポーネント	役割
MidiInputMap / MidiInputRouter	ScriptableObject で定義した MIDI 条件 → UnityEvent 発火
MidiNoteTracker	チャンネルごとの押下ノート状態管理
MidiChannelFilter	特定チャンネルのみ後段へ転送
MidiDeviceFilter	特定デバイスのみ後段へ転送
SmfPlayer	SMF / Sequence の同期再生（詳細は SMF ツール ）
MidiRecorder	ライブ MIDI 入力の SMF 記録（詳細は SMF ツール ）
MidiCcSmoother	CC 値の指数移動平均スムージング
MidiCcButton	CC しきい値によるボタン検出
MidiClockSync	外部 MIDI Clock 同期、BPM / 拍 / 小節提供
MidiClockOutput	MIDI Clock マスター送信
SmfPlayerClockAdapter	外部 Clock に SmfPlayer テンポを追従
MidiChordDetector	和音名・スケール所属の変化検出、目標和音との一致判定（クイズ）

Animator / Timeline / Visual Scripting との統合は [Unity エコシステム統合](#) を参照してください。

いずれも `MidiManager` のイベントハンドラ機構（`IMidi*EventHandler`）を利用します。`InitializeMidi()` の呼び出しはシーン側の責務です。

MidInputMap / MidInputRouter

概要

MidInputMap (ScriptableObject) に MIDI メッセージと UnityEvent のマッピングを定義し、MidInputRouter (MonoBehaviour) が条件一致時にイベントを発火します。Inspector からコードを書かずに MIDI 入力をゲームイベントへ結び付けられます。

セットアップ

- 1. Assets > Create > MIDI > Input Map で MidInputMap を作成
- 2. GameObject に MidInputRouter をアタッチ
- 3. Map フィールドに作成した Input Map を割り当て
- 4. Input Map の Bindings に条件と UnityEvent を設定
- 5. シーン bootstrap で MidiManager.InitializeMidi() を呼び出す

バインディング条件

フィールド	説明
messageType	MidiOutgoingMessageType (NoteOn / CC / PitchWheel / Start / SysEx 等)
group	0-15、-1 = 全 group
deviceIdFilter	空 = 全デバイス
channel	0-15、-1 = 全チャンネル (システム Trigger では無視)
noteNumber	0-127、-1 = 任意 (Note / Poly Aftertouch / システム data byte フィルタ)
minVelocity / maxVelocity	NoteOn のベロシティ範囲
controllerNumber	CC 番号
minValue / maxValue	CC / Aftertouch 値の範囲
ccTriggerMode	CC トリガー方式 (下表)
programNumber	0-127、-1 = 任意 (PC 用)

メッセージ種別とイベント

カテゴリ	messageType 例	発火イベント
演奏系	NoteOn/Off, CC, PitchWheel, Aftertouch	onTriggered + onTriggeredWithValue
演奏系	ProgramChange, SongSelect, ...	onTriggered + onTriggeredWithValue (number)
システム Trigger	TimingClock, Start, Stop, Reset, ...	onTriggered のみ
パイロード	SystemExclusive, SystemCommonMessage, Midi2RawUmp	onMessage のみ

MidiInputRouter 自体にも raiseMessageEvents / onMessage があり、全着信メッセージを DTO (MidiInputMessageEventArgs) で受け取れます。SysEx / Raw UMP の汎用処理に使います。

旧 MidiBindingMessageType アセットは FormerlySerializedAs により自動移行されます (PitchBend → PitchWheel) 。

CC トリガーモード

モード	動作
OnChange	値が範囲内で変化するたび
OnEnterRange	値が minValue 以上になった瞬間
OnExitRange	値が maxValue 以下になった瞬間
OnThreshold	値が minValue 以上になった瞬間 (ボタン用途)

使用例

```
// MidiInputMap を Inspector で設定し、UnityEvent にゲームロジックを接続
// 例: NoteOn ch0 note60 → ライトを点灯
```

CC ボタン (64 以上で 1 回だけ発火) の設定例:

- messageType : ControlChange
- channel : 0
- controllerNumber : 64
- minValue : 64
- ccTriggerMode : OnThreshold

注意点

- 複数バインディングは OR 評価（いずれか一致で発火）
- `onTriggered`（引数なし）と `onTriggeredWithValue`（int 引数）の両方が設定されていれば、両方発火
- フィルタ経由で使用する場合は `registerWithMidiManager = false` に設定（[フィルタとの組み合わせ](#)）

MidiNoteTracker

概要

MIDI チャンネルごとに現在押下中（Note On 済み、Note Off 未受信）のノート番号を管理します。和音判定、同時押し数、キーリリース検出などの基盤として使えます。

設定

フィールド	説明
<code>targetChannels</code>	追跡対象チャンネル（空 = 全チャンネル）
<code>deviceIdFilter</code>	空 = 全デバイス
<code>registerWithMidiManager</code>	<code>MidiManager</code> へ直接登録するか（デフォルト true）
<code>onNoteStateChanged</code>	ノート状態変化時の UnityEvent

API

メソッド	説明
<code>IsNoteOn(channel, note)</code>	指定ノートが押下中か
<code>GetActiveNotes(channel)</code>	押下中ノート一覧
<code>GetActiveNoteCount(channel)</code>	押下中ノート数
<code>GetTotalActiveNoteCount()</code>	全チャンネル合計
<code>HasAnyNoteOn()</code>	いずれかのノートが押下中か

CC 120（All Sound Off）および CC 123（All Notes Off）受信時は、該当チャンネルの全ノートを Off として処理します。

使用例

```
using jp.kshoji.unity.midi;

public sealed class ChordDetector : MonoBehaviour
{
    public MidiNoteTracker tracker;

    void Update()
    {
        if (tracker.GetActiveNoteCount(0) >= 3)
            Debug.Log("3 notes or more pressed on channel 0");
    }
}
```

MidiChannelFilter / MidiDeviceFilter

概要

特定の MIDI チャンネルまたはデバイスのみを通過させ、後段の GameObject にイベントを転送するフィルタコンポーネントです。MIDI 1.0 全メッセージに加え、Raw UMP（IMidi2RawUmpEventHandler）も転送します。UI 用とゲームプレイ用でルートに分ける際に使います。

設定

MidiChannelFilter

フィールド	説明
allowedChannels	許可チャンネル（空 = 全通過）
blockedChannels	ブロックチャンネル（allowed より優先）
forwardTargets	転送先 GameObject 配列

MidiDeviceFilter

フィールド	説明
allowedDeviceIds	許可デバイス ID（空 = 全通過）
blockedDeviceIds	ブロックデバイス ID
allowVirtualDevices	virtual: プレフィックスの仮想デバイスを許可
forwardTargets	転送先 GameObject 配列

フィルタとの組み合わせ

フィルタ経由で後段コンポーネントにイベントを渡す配線例:

```
[MidiManager]
  ↓
[MidiDeviceFilter] ← registerWithMidiManager = true (チェーンの入口)
  ↓ forwardTargets
[MidiChannelFilter] ← registerWithMidiManager = false
  ↓ forwardTargets
[MidiInputRouter]  ← registerWithMidiManager = false
```

重要: フィルタチェーンを使う場合、入口以外の `MidiChannelFilter` は `registerWithMidiManager = false` にし、後段の `MidiInputRouter` や `MidiNoteTracker` も `registerWithMidiManager = false` にしてください。二重登録するとイベントが二重に処理されます。

動作例は `Assets/MIDI/Samples/Gameplay/Scenes/MidiGameplaySampleScene.unity` を参照してください。

フィルタなしで直接使う場合は、Router / Tracker の `registerWithMidiManager = true` (デフォルト) のままで問題ありません。

MidiClockSync / 和音・スケール判定

MidiClockSync

外部シーケンサー等からの MIDI Clock に同期します。BPM・拍・小節イベントをゲームロジックや `SmfPlayerClockAdapter` から利用できます。

プロパティ / API	説明
<code>pulsesPerQuarterNote</code>	1 拍あたりの Clock パルス数 (MIDI 標準 24)
<code>beatsPerBar</code>	小節あたりの拍数 (既定 4)
<code>deviceIdFilter</code>	空 = 全デバイス
<code>EstimatedBpm</code> / <code>IsBpmStable</code>	BPM 推定値と安定フラグ
<code>onBeat</code> / <code>onBar</code>	拍 / 小節境界
<code>onStarted</code> / <code>onStopped</code>	Start / Stop 受信

```
var clock = gameObject.AddComponent<MidiClockSync>();
clock.onBeat.AddListener(() => Debug.Log("Beat"));
clock.onBar.AddListener(() => Debug.Log("Bar"));
```

`MidiClockOutput` で 120 BPM の Clock を送信できます。`SmfPlayerClockAdapter` は `Follow` / `Step` / `Free` モードで `SmfPlayer` のテンポまたは再生位置を外部 Clock に連携します。

和音・スケール判定

`MidiNoteTracker` と組み合わせて和音名推定・スケール外ノート検出を行います。

```
using jp.kshoji.unity.midi.util;

var notes = noteTracker.GetActiveNotes(0);
var chord = ChordRecognition.Recognize(notes);
var inScale = ScaleUtility.AllNotesInScale(notes, rootNote: 0, MidiScaleType.Major);
```

`MidiChordDetector` は和音変化を `onChordChanged` で通知し、スケール外入力を `onScaleViolation` で検出します。`targetChord` を設定して `EvaluateNow()` を呼ぶと、目標和音との一致判定 (クイズ) が `onQuizCorrect` / `onQuizIncorrect` で通知されます。`onAllNotesInScale` は押下ノートがすべてスケール内のときに発火します。

MidiCcSmoother / MidiCcButton

CC 値のスムージングとしきい値トグルを提供します。Animator 統合などからも利用できます。

MidiCcProcessorBase

CC プロセッサコンポーネントの抽象基底。チャンネル / デバイス照合ヘルパーと、EMA / 時間ベースのスムーズヘルパー（EMA は `MidiControlSmoothing` に委譲）。

場所: `Assets/MIDI/Scripts/Gameplay/MidiCcProcessorBase.cs`

API	説明
<code>MatchesChannel</code> / <code>MatchesDevice</code>	フィルタヘルパー（-1 / 空 = すべて一致）
<code>ApplyEma(...)</code>	<code>MidiControlSmoothing.ApplyEma</code> に委譲
<code>SmoothTowards(current, target, smoothingTime, deltaTime)</code>	正規化ターゲットへ向かうフレームベースの lerp

MidiCcSmoother

項目	説明
<code>configs[]</code>	コントローラー番号・チャンネル・ <code>smoothFactor</code> （0.01-1.0、小さいほど滑らか）
<code>onSmoothedValue[]</code>	正規化値 (0-1) の UnityEvent
<code>GetSmoothedNormalizedValue(index)</code>	最新の正規化値を取得

MidiCcButton

項目	説明
<code>onThreshold</code> / <code>offThreshold</code>	ヒステリシス付きしきい値（既定 64 / 63）
<code>onPressed</code> / <code>onReleased</code>	押下 / 解放イベント
<code>IsPressed</code>	現在のオン状態

ソースファイル

ファイル	役割
MidiInputMap.cs	ScriptableObject 定義
MidiInputBinding.cs	バインディング定義
MidiInputCondition.cs	条件評価ロジック
MidiInputBindingMigration.cs	旧 enum 移行
MidiInputMessageEventArgs.cs	onMessage DTO
MidiInputRouter.cs	イベントルーター
MidiNoteTracker.cs	ノート状態管理
MidiNoteStateChangedEvent.cs	状態変化イベント引数
MidiFilterBase.cs	フィルタ基底クラス
MidiChannelFilter.cs	チャンネルフィルタ
MidiDeviceFilter.cs	デバイスフィルタ
MidiEventForwarder.cs	イベント転送ヘルパー
SmfPlayer.cs / SmfPlayerState.cs	SMF 再生
MidiRecorder.cs	SMF 録音
MidiClockSync.cs / MidiClockState.cs / MidiClockOutput.cs	MIDI Clock 同期
SmfPlayerClockAdapter.cs	SMF と外部 Clock のテンポ連携
MidiChordDetector.cs	和音・スケール判定（リアルタイム検出 + クイズ）
MidiCcProcessorBase.cs	CC プロセッサの共通基底
MidiCcSmoother.cs / MidiCcButton.cs	CC スムージング・しきい値トグル
Gameplay/Theory/ChordRecognition.cs / Gameplay/Theory/ScaleUtility.cs	和音名推定・スケールユーティリティ
MidiManagerReceiverBridge.cs	Sequencer → MidiManager ブリッジ

関連ドキュメント

- [MIDI 1.0 ランタイム \(MidiManager\)](#)
- [ユーティリティ \(MidiNoteUtility / MidiMessageBuilder\)](#)
- [エディタツール \(MIDI モニター\)](#)
- [SMF ツール \(SmfPlayer / MidiRecorder\)](#)
- [スタートガイド](#)